

Projet réalisé par

Sirius Dos Santos

Senthalan Thayaparan

Thomas David

Master CDSI

2025-2026

Détection d'Intrusions Réseau par Machine Learning

Matière : Cryptographie et algorythmie

Professeur : Moad Dehbi

Mars 2026

Table des matières

1	Introduction	4
1.1	Contexte et enjeux	4
1.2	Systèmes de détection d'intrusions	4
1.3	Objectif	5
2	Le Dataset NSL-KDD	6
2.1	Présentation	6
2.2	Structure des données	6
2.3	Déséquilibre des classes	6
3	Préparation des Données	8
3.1	Encodage des variables catégorielles	8
3.2	Normalisation (StandardScaler)	8
3.3	Traitement des valeurs aberrantes	8
3.4	Rééquilibrage avec SMOTE	9
4	Modèles de Machine Learning	11
4.1	Protocole expérimental	11
4.2	Présentation des algorithmes	11
4.2.1	Random Forest	11
4.2.2	SVM, Support Vector Machine	11
4.2.3	MLP, Perceptron Multicouches	12
4.2.4	XGBoost	12
4.2.5	LightGBM	12
5	Résultats et Analyse	13
5.1	Métriques d'évaluation	13
5.2	Résultats globaux	13
5.3	Rappel par classe	14
5.4	Analyse des matrices de confusion	15
5.5	Courbes ROC	20
6	Interprétabilité, Analyse SHAP	21
6.1	Motivation	21
6.2	Méthode SHAP	21
6.3	Résultats	22
7	Modèle Hybride CNN-LSTM	26

7.1	Motivation	26
7.2	Architecture	26
7.2.1	Bloc 1, CNN 1D	26
7.2.2	Bloc 2, LSTM Bidirectionnel	26
7.2.3	Bloc 3, Mécanisme d'Attention	27
7.3	Régularisation et entraînement	27
7.4	Résultats et limitations	27
8	Limites et Perspectives	29
8.1	Limites	29
8.1.1	Dataset obsolète	29
8.1.2	Biais de SMOTE	29
8.1.3	Classes rares insuffisamment apprises	29
8.2	Perspectives	29
8.2.1	Données	29
8.2.2	Modèles	30
8.2.3	Déploiement	30
8.2.4	Contre le déséquilibre	30
9	Conclusion	31
A	Manuel de déploiement	32
A.1	Installation via Docker	32
A.2	Structure du projet	32
A.3	Exécution	32
A.4	Fichiers générés	33

1. Introduction

TABLE DES MATIÈRES *Rédacteur : Dos Santos Sirius*

1.1 Contexte et enjeux

Les infrastructures informatiques modernes sont la cible d'une activité malveillante croissante et structurée. À l'échelle mondiale, plus de 2 200 cyberattaques se produisent chaque jour, et le coût moyen d'une violation de données s'élève à 4,45 millions de dollars (IBM Security, 2023). Les entreprises mettent en moyenne 207 jours avant de détecter qu'elles ont été compromises, délai pendant lequel les attaquants peuvent exfiltrer des données, installer des portes dérobées ou préparer des attaques de plus grande envergure.

Face à ce constat, les solutions de sécurité périmétrique traditionnelles (pare-feux, antivirus à base de signatures) montrent leurs limites : elles ne détectent que les menaces qu'elles ont déjà répertoriées. Les attaques zero-day, c'est-à-dire exploitant des vulnérabilités inconnues, les contournent systématiquement.

1.2 Systèmes de détection d'intrusions

Définition, IDS, Intrusion Detection System

Un IDS est un système logiciel qui surveille en continu le trafic réseau ou les activités d'un système, et qui génère une alerte lorsqu'il détecte un comportement correspondant à une intrusion ou à une anomalie.

On distingue deux grandes familles d'IDS selon leur périmètre d'observation :

- **NIDS** (Network Intrusion Detection System) : surveille le trafic circulant sur un segment réseau. Il analyse les paquets au niveau du réseau et peut ainsi détecter des attaques affectant plusieurs machines. C'est l'approche adoptée dans ce projet.
- **HIDS** (Host Intrusion Detection System) : installé directement sur un hôte, il surveille les appels système, les fichiers de log et les accès aux ressources locales.

Et deux approches de détection :

Approche	Principe	Limite
Signatures	Compare le trafic à une base de menaces connues	Aveugle aux nouvelles attaques
Comportements (ML)	Apprend ce qui est normal et détecte les écarts	Requiert de nombreuses données d'entraînement

Ce projet implémente la seconde approche : un IDS comportemental basé sur le Machine Learning.

1.3 Objectif

L'objectif est de construire un système capable de classifier automatiquement chaque connexion réseau en cinq catégories représentant les types d'attaques les plus courants :

Classe	Type d'attaque	Exemple concret
Normal	Trafic légitime	Navigation web, email
DoS	Déni de service	Inondation SYN, ping flood
Probe	Reconnaissance réseau	Scan de ports (nmap)
R2L	Accès distant non autorisé	Brute force SSH, FTP
U2R	Escalade de privilèges	Exploitation d'un buffer overflow

2. Le Dataset NSL-KDD

Rédacteur : Dos Santos Sirius

2.1 Présentation

Définition, NSL-KDD

NSL-KDD est un dataset de référence pour la recherche en détection d'intrusions, publié en 2009 par Tavallae et al. Il est construit à partir de captures de trafic réseau réel et corrige plusieurs défauts du dataset KDD Cup 1999 original, notamment la présence de doublons qui biaisaient artificiellement les résultats.

Les fichiers sont téléchargés automatiquement depuis des miroirs GitHub par le module `src/download_data.py`. Aucune intervention manuelle n'est requise.

Fichier	Usage	Connexions
KDDTrain+.txt	Entraînement des modèles	125 973
KDDTest+.txt	Évaluation finale	22 544

2.2 Structure des données

Chaque ligne du dataset représente une connexion réseau décrite par **41 features**. Ces features sont de deux types :

- **38 features numériques** : durée de la connexion, volume de données échangées, nombre de connexions vers la même destination dans les dernières secondes, etc.
- **3 features catégorielles** : `protocol_type` (TCP, UDP, ICMP), `service` (70 valeurs : http, ftp, smtp, ...), `flag` (11 états de connexion TCP).

2.3 Déséquilibre des classes

Le déséquilibre des classes est le problème central du dataset. Il est présent aussi bien dans l'ensemble d'entraînement que dans l'ensemble de test, mais avec des proportions différentes :

Classe	Train	% train	Test	% test
Normal	67 343	53,5 %	9 711	43,1 %
DoS	45 927	36,5 %	7 460	33,1 %
Probe	11 656	9,3 %	2 421	10,7 %
R2L	995	0,8 %	2 885	12,8 %
U2R	52	0,04 %	67	0,3 %
Total	125 973		22 544	

Deux observations importantes découlent de ce tableau. D'une part, la classe U2R ne compte que 52 exemples en entraînement, un modèle entraîné sans traitement spécifique ne les verra presque jamais et ne pourra pas les apprendre. D'autre part, la distribution du jeu de test est délibérément différente de celle du jeu d'entraînement : R2L représente 0,8 % de l'entraînement mais 12,8 % du test. Cette asymétrie simule les conditions réelles, où les attaques rares apparaissent en rafale lors d'une campagne ciblée.

3. Préparation des Données

Rédacteur : Dos Santos Sirius

Le preprocessing transforme les données brutes en un format exploitable par les algorithmes de ML. Il comporte quatre étapes séquentielles.

3.1 Encodage des variables catégorielles

Les algorithmes de ML opèrent sur des espaces vectoriels réels. Les variables catégorielles doivent donc être converties en représentations numériques. On utilise le **OneHotEncoding** : pour une variable à k modalités, on crée k colonnes binaires dont une seule vaut 1 à la fois.

Note

Les 3 features catégorielles (`protocol_type`, `service`, `flag`) génèrent $3 + 70 + 11 = 84$ colonnes binaires. Le vecteur de features passe donc de 41 à 122 dimensions après encodage.

3.2 Normalisation (StandardScaler)

Les features numériques présentent des ordres de grandeur très hétérogènes : `duration` varie entre 0 et quelques milliers de secondes, tandis que `src_bytes` peut atteindre plusieurs milliards pour des attaques DoS. Sans normalisation, les features à grande variance dominant artificiellement la fonction de coût.

La normalisation Z-score transforme chaque feature x selon :

$$\tilde{x} = \frac{x - \mu}{\sigma}$$

où μ est la moyenne et σ l'écart-type calculés sur l'ensemble d'entraînement uniquement (pour éviter toute fuite d'information vers le test).

3.3 Traitement des valeurs aberrantes

La méthode **IQR** (Interquartile Range) identifie les valeurs aberrantes au-delà des seuils $[Q_1 - 1,5 \cdot \text{IQR}, Q_3 + 1,5 \cdot \text{IQR}]$. Ces valeurs sont clippées (ramenées au seuil) plutôt que supprimées, pour ne pas réduire le dataset.

3.4 Rééquilibrage avec SMOTE

Définition, SMOTE, Synthetic Minority Oversampling Technique

SMOTE génère de nouveaux exemples synthétiques pour les classes minoritaires en interpolant entre des exemples réels proches dans l'espace des features. Pour chaque exemple minoritaire, l'algorithme sélectionne aléatoirement un de ses k plus proches voisins (même classe) et génère un nouvel exemple sur le segment qui les relie.

Concrètement, pour la classe U2R (52 exemples initiaux), SMOTE en génère 67 291 supplémentaires pour atteindre 67 343, soit une multiplication par 1 295. Le même processus s'applique à toutes les classes sous-représentées.

	Avant SMOTE	Après SMOTE
Nombre d'exemples	125 973	336 715
Distribution	déséquilibrée	équilibrée (67 343/classe)

Note, Limite de SMOTE

SMOTE améliore considérablement les scores de validation sur données équilibrées ($F1 \approx 0,999$ dans notre cas). Cependant, les exemples synthétiques ne capturent pas nécessairement la vraie variabilité des attaques rares. Sur le jeu de test réel (déséquilibré, distribution différente), les performances chutent à $F1 \approx 0,72$. Cet écart est une leçon fondamentale sur la différence entre performance en validation et performance en production.

La figure 3.1 présente la distribution des classes avant et après SMOTE. La barre U2R, quasi-invisible avant traitement, atteint le même niveau que les autres classes après SMOTE.

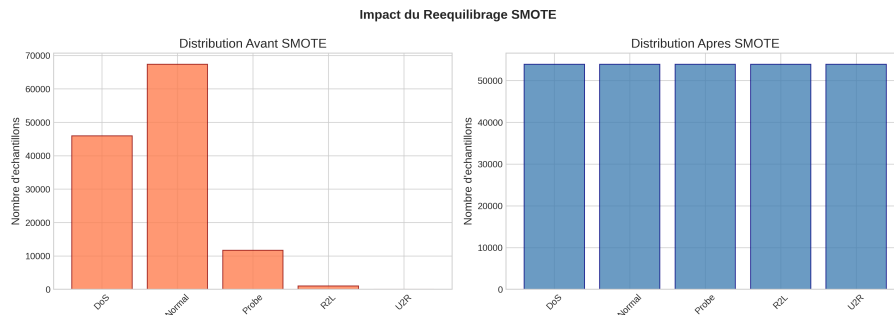


Figure 3.1. Distribution des classes avant et après rééquilibrage SMOTE. Avant traitement, U2R (52 exemples) est invisible à cette échelle face à Normal (67 343). Après SMOTE, toutes les classes sont représentées par 67 343 exemples.

La figure 3.2 projette le dataset d'entraînement en deux dimensions par Analyse en Composantes Principales (PCA). Cette réduction de dimension permet de visualiser la structure des données, bien que 41 dimensions ne puissent être fidèlement représentées en 2D.



Figure 3.2. Projection PCA 2D des données d'entraînement après preprocessing. Les classes DoS et Normal forment des nuages distincts et bien séparés, ce qui explique les bons rappels sur ces classes. Les points R2L et U2R, peu nombreux même après SMOTE à cette échelle, se confondent avec les autres classes, illustration de la difficulté structurelle de leur détection.

4. Modèles de Machine Learning

Rédacteur : Senthalan Thayaparan

4.1 Protocole expérimental

Les données après SMOTE (336 715 exemples) sont divisées en trois ensembles : 80 % pour l'entraînement (269 372 exemples), 20 % pour la validation (67 343 exemples), et le jeu de test officiel NSL-KDD (22 544 exemples) pour l'évaluation finale. La stratification garantit le maintien des proportions entre les splits.

Cinq algorithmes sont entraînés. Les hyperparamètres peuvent être optimisés automatiquement par **Optuna** (stratégie TPE, 30 essais, objectif : F1-score macro). Dans nos expériences, l'optimisation est désactivée (`-no-optimize`) pour réduire le temps de calcul.

4.2 Présentation des algorithmes

4.2.1 Random Forest

Définition, Forêt aléatoire

Un ensemble de T arbres de décision entraînés sur des sous-ensembles aléatoires des données (bagging) et des features. La prédiction finale est le vote majoritaire des T arbres.

Paramètres utilisés : $T = 100$ arbres, critère Gini, profondeur non limitée. L'entraînement est parallélisé sur tous les cœurs disponibles (`n_jobs=-1`). Temps d'entraînement : **3,7 s**.

4.2.2 SVM, Support Vector Machine

Définition, SVM linéaire

Le SVM cherche l'hyperplan qui maximise la marge entre les classes dans un espace de features transformé. Pour la classification multiclasse, on utilise la stratégie one-vs-rest.

Implémentation : `LinearSVC` enveloppé dans `CalibratedClassifierCV(cv=3)`, qui entraîne le classifieur trois fois en validation croisée pour calibrer les probabilités de sortie. Cette calibration est nécessaire pour le calcul de l'AUC-ROC mais multiplie par trois le temps d'entraînement. Sur 336 715 exemples : **1 343 s** (22 minutes).

4.2.3 MLP, Perceptron Multicouches

Définition, MLP

Un réseau de neurones à propagation avant composé de couches entièrement connectées. L'apprentissage s'effectue par rétropropagation du gradient et optimisation stochastique.

Configuration : architecture 3 couches cachées, activation ReLU, optimiseur Adam, régularisation L2. Temps d'entraînement : **184 s**.

4.2.4 XGBoost

Définition, Gradient Boosting

Méthode d'ensemble qui construit les modèles séquentiellement : chaque nouvel arbre corrige les erreurs résiduelles du modèle précédent en minimisant une fonction de perte régularisée.

XGBoost est une implémentation optimisée du gradient boosting, avec parallélisation des splits, régularisation L1/L2 et gestion native des valeurs manquantes. Temps d'entraînement : **6,7 s**.

4.2.5 LightGBM

LightGBM est une variante d'XGBoost utilisant la croissance des arbres par feuille (leaf-wise) plutôt que par niveau (level-wise), et le binning des features contiguës pour accélérer le calcul des splits. Temps d'entraînement : **10,1 s**.

5. Résultats et Analyse

Rédacteur : Senthalan Thayaparan

5.1 Métriques d'évaluation

Trois métriques complémentaires sont utilisées :

- **Accuracy** : proportion de connexions correctement classifiées. Insuffisante seule sur des classes déséquilibrées.
- **F1-Score macro** : moyenne non pondérée des F1-scores par classe. Pénalise fortement les mauvaises performances sur les classes rares.
- **AUC-ROC** : aire sous la courbe ROC, calculée en stratégie one-vs-rest. Mesure la capacité discriminante indépendamment du seuil de décision.

5.2 Résultats globaux

Le tableau 5.2 présente les performances sur le jeu de test officiel NSL-KDD (22 544 connexions).

Modèle	Accuracy	F1-Score	AUC-ROC	Train (s)
MLP	0,761	0,726	0,936	184
XGBoost	0,761	0,723	0,963	7
LightGBM	0,751	0,716	0,955	10
SVM	0,743	0,733	0,917	1 343
Random Forest	0,734	0,687	0,949	4
CNN-LSTM*	0,431	0,259	0,564	,

* Non entraîné (**-skip-hybrid**), poids aléatoires. Voir chapitre 7.

Table 5.1. Performances sur le jeu de test NSL-KDD (22 544 connexions). Les scores de validation sur données SMOTE équilibrées étaient de $F1 \approx 0,999$ pour tous les modèles classiques.

Plusieurs observations méritent d'être soulignées. MLP et XGBoost atteignent la même accuracy (0,761), mais XGBoost dispose du meilleur AUC-ROC (0,963). Le SVM, malgré une accuracy inférieure, obtient le meilleur F1-Score global (0,733), il répartit mieux ses erreurs entre les classes plutôt que de tout concentrer sur les classes dominantes. Random Forest, pourtant très rapide (3,7 s), est le moins précis.

5.3 Rappel par classe

Le tableau 5.2 détaille le rappel (recall) par classe, qui est la métrique la plus pertinente en sécurité : un rappel faible sur U2R signifie que la majorité des escalades de privilèges passent inaperçues.

Modèle	Normal	DoS	Probe	R2L	U2R
SVM	0,90	0,74	0,72	0,25	0,49
MLP	0,92	0,85	0,68	0,08	0,21
XGBoost	0,97	0,78	0,70	0,06	0,16
LightGBM	0,97	0,77	0,64	0,08	0,12
Random Forest	0,97	0,76	0,60	0,01	0,12

Table 5.2. Rappel par classe sur le jeu de test. Valeurs en gras : meilleur score par colonne.

L'analyse par classe révèle un compromis fondamental. XGBoost, LightGBM et Random Forest excellent sur Normal (0,97) et DoS (0,76–0,78), mais sont pratiquement aveugles à R2L (recall $\leq 0,08$). Le SVM présente un profil plus équilibré : son recall U2R de 0,49 est le double de ses concurrents, au prix d'un recall Normal légèrement inférieur (0,90). Ce compromis précision/rappel sur les classes rares est caractéristique de la différence de marge d'apprentissage entre SVM et méthodes d'ensemble.

5.4 Analyse des matrices de confusion

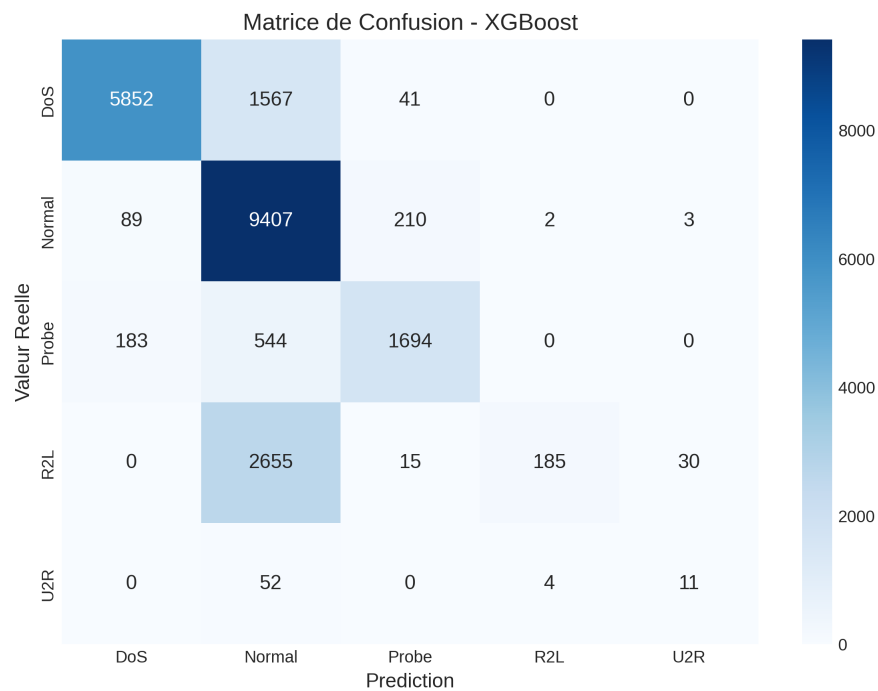


Figure 5.1. Matrice de confusion, XGBoost. Les cellules de la diagonale (prédictions correctes) montrent un excellent recall sur Normal et DoS. On observe une confusion significative entre R2L et Normal : une grande partie des intrusions distantes sont classées comme trafic légitime. La classe U2R, avec seulement 67 exemples dans le test, n'est quasiment jamais détectée.

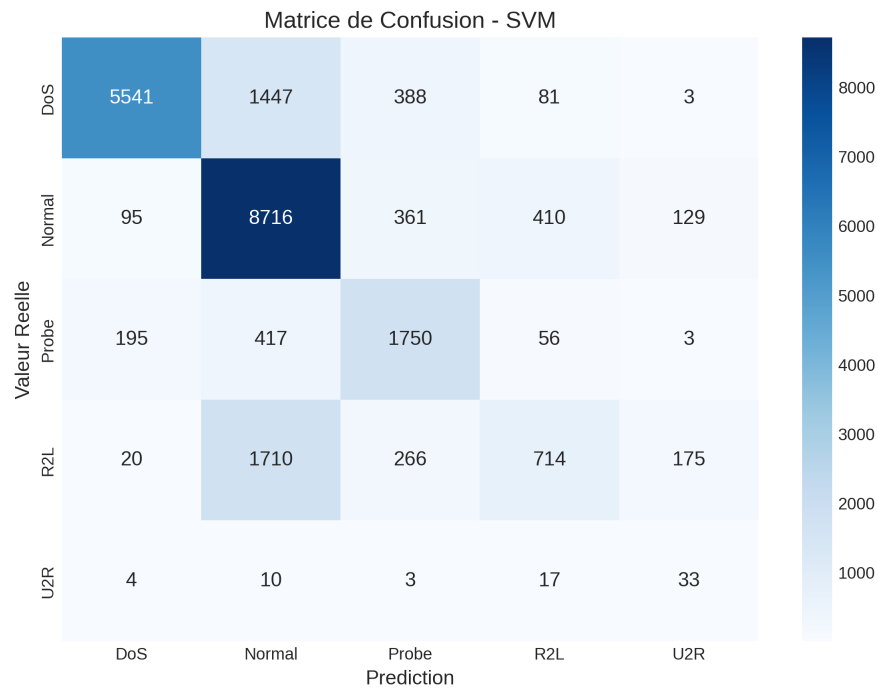


Figure 5.2. Matrice de confusion, SVM. Comparé à XGBoost, le SVM détecte davantage d'exemples U2R (environ la moitié) et R2L, au prix d'un léger affaiblissement sur Normal et DoS. Ce comportement confirme que le SVM, par sa maximisation de la marge, est intrinsèquement plus sensible aux classes minoritaires.

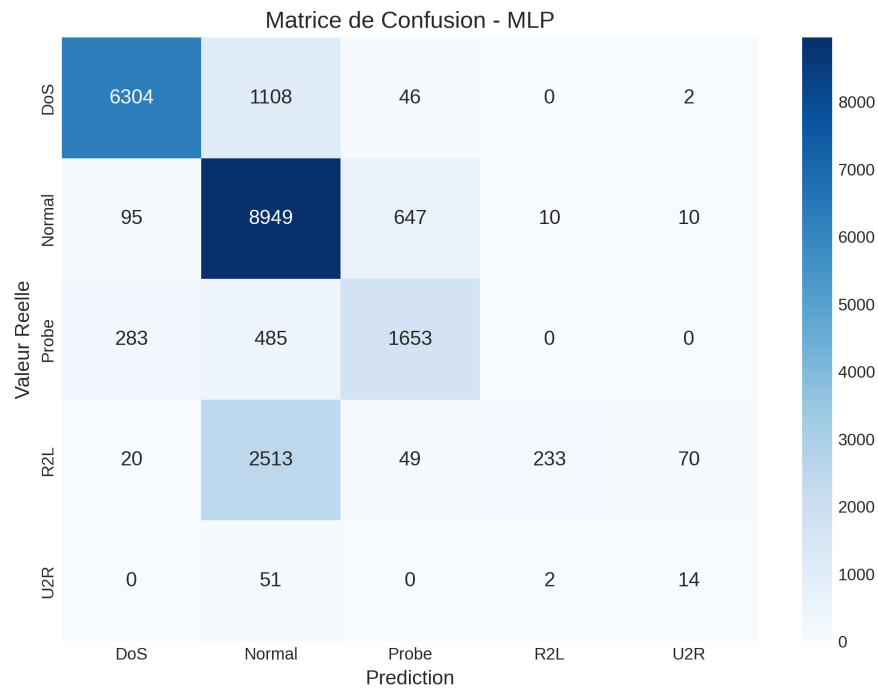


Figure 5.3. Matrice de confusion, MLP. Le réseau de neurones obtient le meilleur recall DoS (0,85) mais est très faible sur R2L et U2R. Les confusions DoS/Normal et Probe/Normal sont les plus fréquentes, suggérant que le MLP généralise bien sur les classes dominantes mais n'a pas suffisamment appris les patterns des attaques rares.

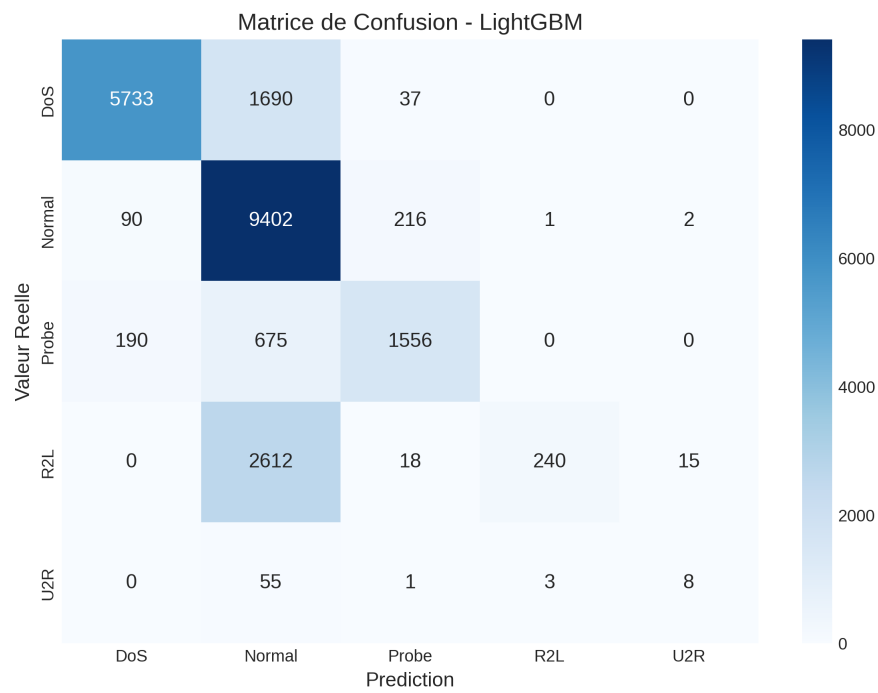


Figure 5.4. Matrice de confusion, LightGBM. Profil très similaire à XGBoost : excellent sur Normal et DoS, quasi-nul sur R2L. LightGBM est légèrement moins performant qu’XGBoost sur Probe (0,64 contre 0,70), mais 1,5 fois plus rapide à l’entraînement.

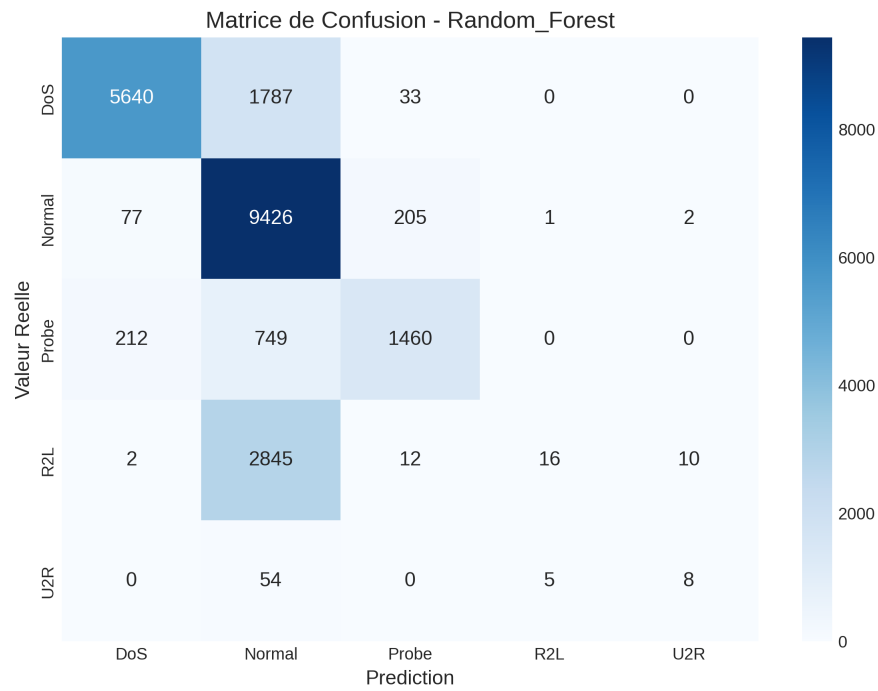


Figure 5.5. Matrice de confusion, Random Forest. Le recall sur R2L est quasi-nul (0,01) : le modèle classe la quasi-totalité des intrusions distantes comme Normal. Malgré son AUC-ROC élevé (0,949), Random Forest est le modèle le moins adapté à la détection d'intrusions rares dans cette configuration.

5.5 Courbes ROC

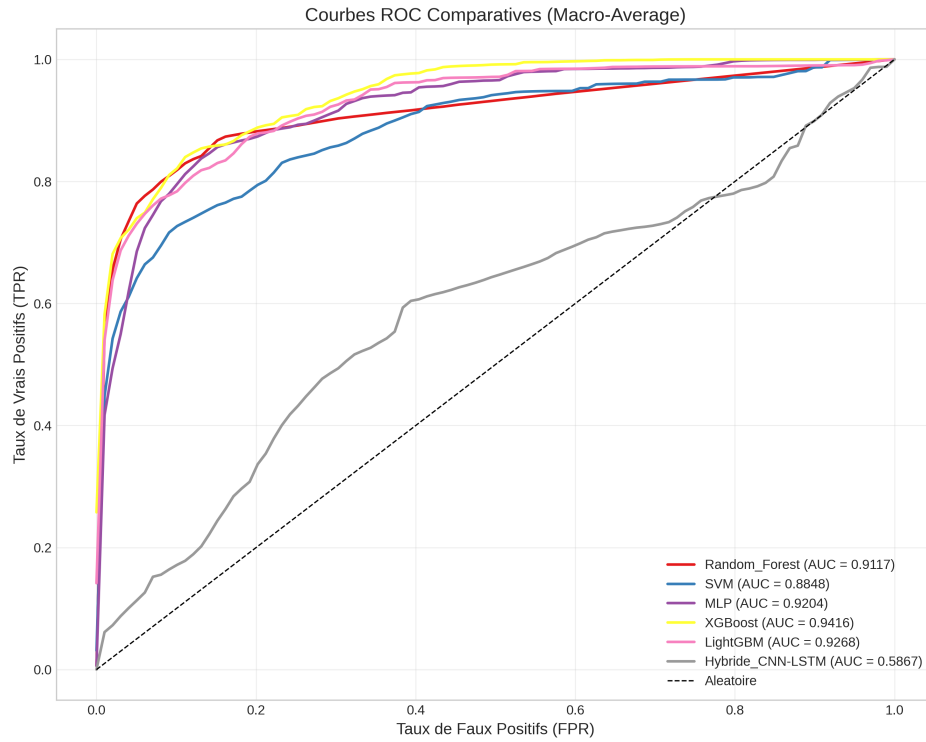


Figure 5.6. Courbes ROC comparatives (stratégie one-vs-rest, moyenne macro). XGBoost (AUC = 0,963) et LightGBM (AUC = 0,955) dominent nettement. Ces AUC élevées signifient que les modèles disposent d'une bonne capacité discriminante globale, mais que les seuils de décision par défaut ne sont pas optimaux pour les classes rares. En production, un ajustement des seuils par classe (notamment augmenter la sensibilité pour U2R au prix de plus de faux positifs) serait nécessaire.

6. Interprétabilité, Analyse SHAP

Rédacteur : Senthalan Thayaparan

6.1 Motivation

Un modèle de Machine Learning peut atteindre de bonnes performances sans que l'on comprenne pourquoi il prend telle ou telle décision. En sécurité, cette opacité est problématique : un analyste SOC qui reçoit une alerte doit pouvoir en évaluer la pertinence, identifier les features suspectes, et décider d'une réponse adaptée. Un modèle « boîte noire » ne lui fournit pas ces éléments.

6.2 Méthode SHAP

Définition, SHAP, SHapley Additive exPlanations

SHAP est une méthode d'interprétabilité post-hoc fondée sur les valeurs de Shapley de la théorie des jeux coopératifs. Pour chaque prédiction et chaque feature, SHAP calcule la contribution marginale de cette feature à la déviation entre la prédiction et la prédiction moyenne du modèle.

La valeur SHAP ϕ_i d'une feature i est définie comme la moyenne pondérée de ses contributions marginales sur toutes les coalitions de features possibles :

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

où F est l'ensemble de toutes les features et $f(S)$ la prédiction du modèle en n'utilisant que les features de S . L'analyse est conduite sur XGBoost, avec 500 exemples du jeu de test.

6.3 Résultats

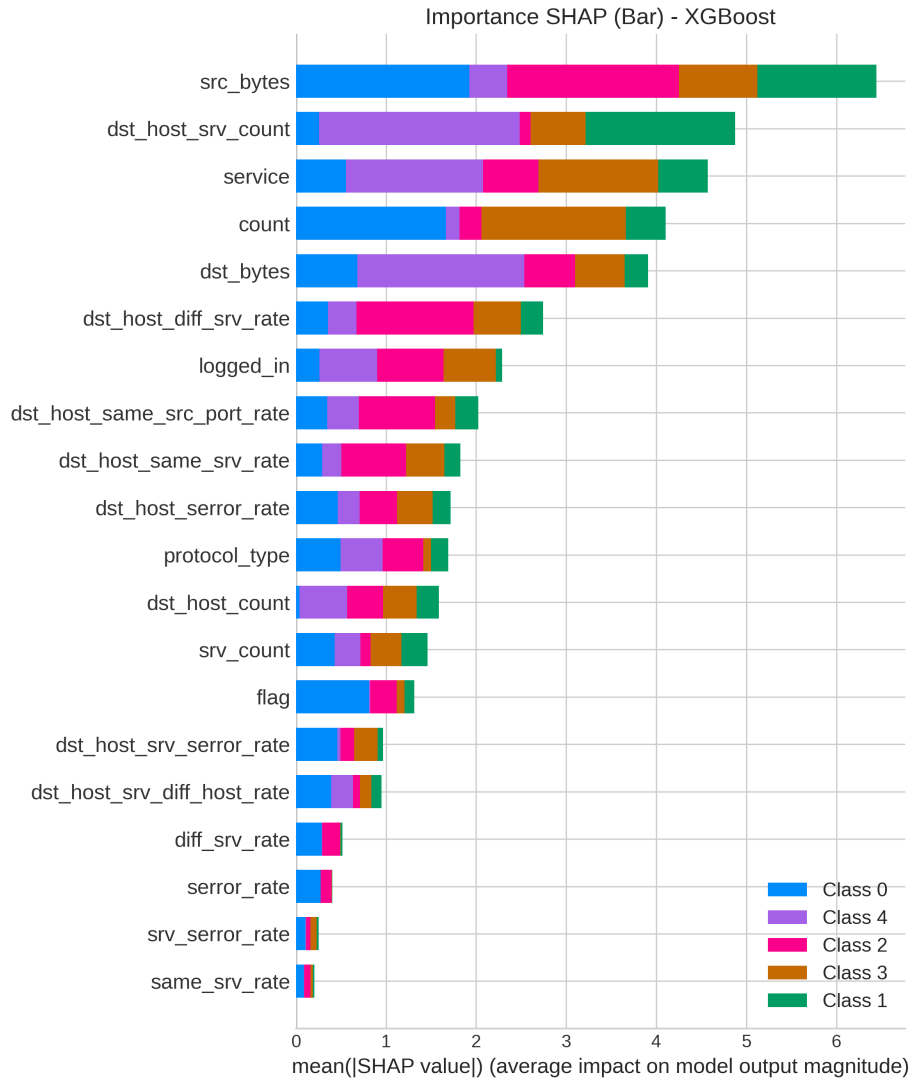


Figure 6.1. Importance globale des features (SHAP, valeur absolue moyenne). Les cinq features les plus discriminantes sont **src_bytes**, **dst_host_srv_count**, **service**, **flag** et **count**. L'importance décroît rapidement au-delà du top 5, ce qui indique une forte concentration de l'information discriminante dans un sous-ensemble restreint de features.

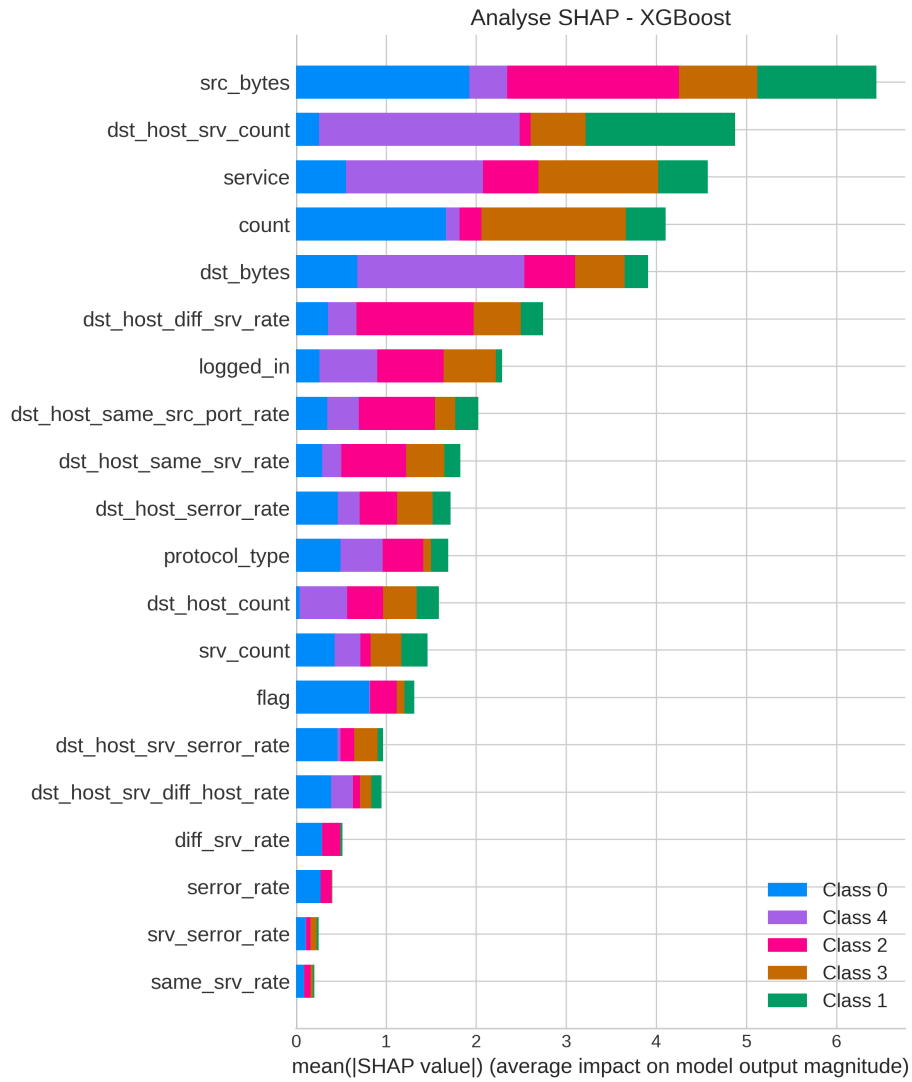


Figure 6.2. Summary plot SHAP (dot plot). Chaque point représente un exemple ; sa position horizontale indique la valeur SHAP (impact sur la prédiction), sa couleur la valeur de la feature (rouge = élevé, bleu = faible). Pour **src_bytes** : les valeurs élevées (rouge) poussent fortement la prédiction vers DoS (valeurs SHAP positives), confirme le pattern d'inondation par volume. Pour **dst_host_srv_count** : la diversité élevée des services contactés est un signal fort de Probe (scan de ports). La feature **flag** montre que l'état S0 (connexion initiée sans réponse) est un indicateur d'anomalie de protocole.

Le tableau 6.1 synthétise l'interprétation des features les plus importantes :

Feature	Description	Signal d'attaque
src_bytes	Volume de données envoyées par la source	Élevé → DoS
dst_host_srv_count	Nombre de services distincts contactés	Élevé → Probe
service	Type de service ciblé	http/ftp → R2L
flag	État de la connexion TCP	S0 → anomalie
count	Connexions récentes vers la même IP	Élevé → DoS
dst_host_same_srv_rate	Taux de connexions vers le même service	Faible → Probe

Table 6.1. Interprétation des six features les plus importantes selon SHAP.

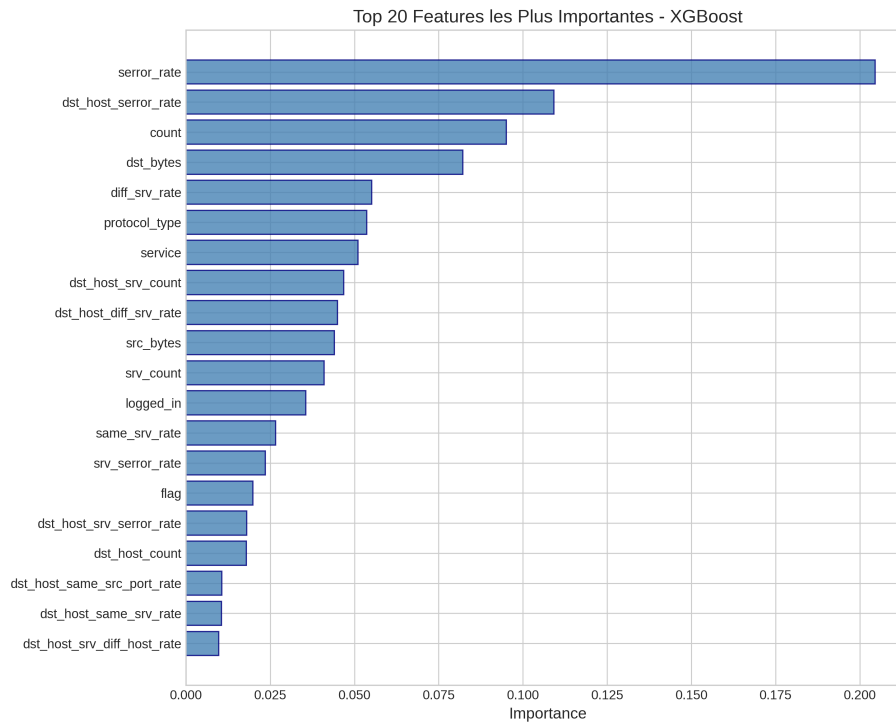


Figure 6.3. Importance des features par gain d'information (XGBoost). Cohérente avec l'analyse SHAP : **src_bytes** domine, suivi de features décrivant le comportement réseau agrégé. Les features de bas niveau (flags TCP, type de protocole) contribuent moins que les statistiques de flux.

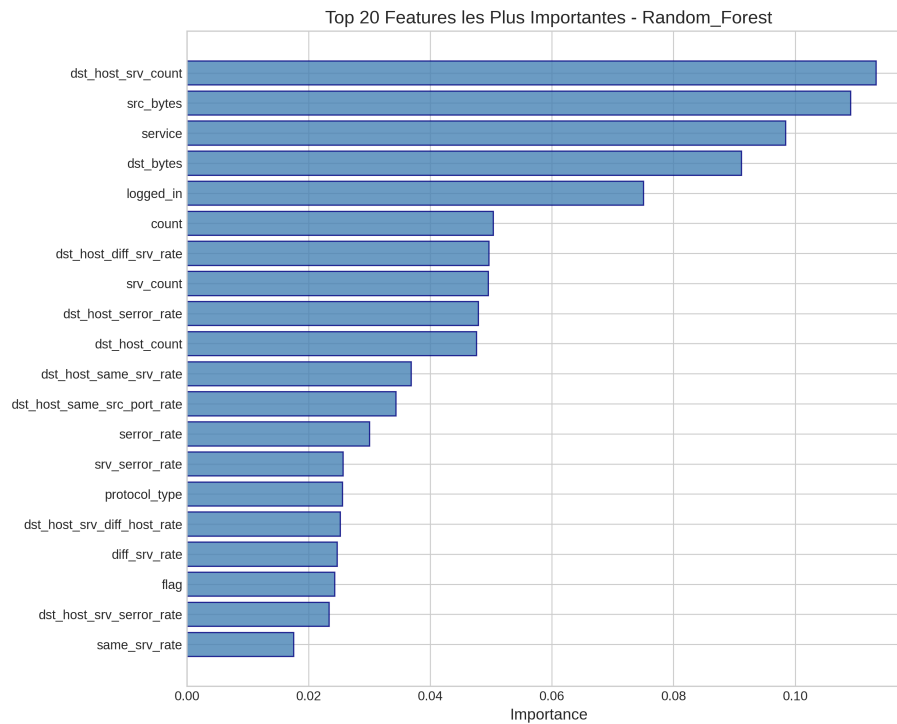


Figure 6.4. Importance des features (Random Forest, impureté de Gini). Random Forest et XGBoost s'accordent sur le top 5, ce qui valide la robustesse de ces features comme indicateurs discriminants indépendamment de l'algorithme utilisé. Cette convergence augmente la confiance dans leur pertinence pour la détection d'intrusions.

7. Modèle Hybride CNN-LSTM

Rédacteur : Thomas David

7.1 Motivation

Les modèles classiques traitent chaque connexion de manière indépendante. Or, en conditions réelles, de nombreuses attaques se déroulent en plusieurs étapes consécutives : une phase de reconnaissance (Probe), suivie d'une tentative d'intrusion (R2L), puis d'une escalade de privilèges (U2R). Un modèle capable d'analyser des *séquences* de connexions peut détecter ces patterns temporels invisibles à une analyse connexion par connexion.

7.2 Architecture

Le modèle hybride combine trois blocs en série pour un total de **240 902 paramètres entraînables**.

7.2.1 Bloc 1, CNN 1D

Définition, CNN 1D appliqué aux features

Une couche de convolution 1D appliquée à un vecteur de features recherche des patterns locaux entre features adjacentes, similairement à la détection de contours en vision par ordinateur.

Architecture : 2 couches convolutionnelles ($64 \rightarrow 128$ filtres, noyau de taille 3) suivies de BatchNorm et MaxPooling. L'idée est de détecter des co-occurrences locales dans l'espace des features : par exemple, `src_bytes` élevé simultanément à `flag=S0`.

7.2.2 Bloc 2, LSTM Bidirectionnel

Définition, LSTM, Long Short-Term Memory

Un LSTM est un réseau récurrent doté de portes d'entrée, d'oubli et de sortie qui régulent le flux d'information à travers le temps, permettant de mémoriser des dépendances à long terme.

Architecture : 2 couches LSTM bidirectionnelles (`hidden_size = 64`). La bidirectionnalité permet de lire la séquence de connexions dans les deux sens, ce qui améliore la détection de patterns dont le signal précède ou suit la connexion anormale.

7.2.3 Bloc 3, Mécanisme d'Attention

Définition, Attention

Le mécanisme d'attention calcule un vecteur de poids α_t pour chaque position temporelle de la séquence, permettant au modèle de pondérer différemment les connexions selon leur pertinence pour la classification finale.

La sortie du LSTM h_t est transformée en score d'attention $e_t = \tanh(Wh_t + b)$, normalisé par softmax : $\alpha_t = \text{softmax}(e_t)$. La représentation finale est la somme pondérée $c = \sum_t \alpha_t h_t$.

7.3 Régularisation et entraînement

- **Dropout** (0,3 en entrée, 0,5 avant la couche de classification) : désactivation aléatoire de neurones à l'entraînement pour forcer une représentation distribuée.
- **Early Stopping** (patience = 10 epochs) : arrêt si le F1 de validation ne s'améliore plus.
- **ReduceLROnPlateau** : division du taux d'apprentissage par 2 si la loss de validation stagne.

7.4 Résultats et limitations

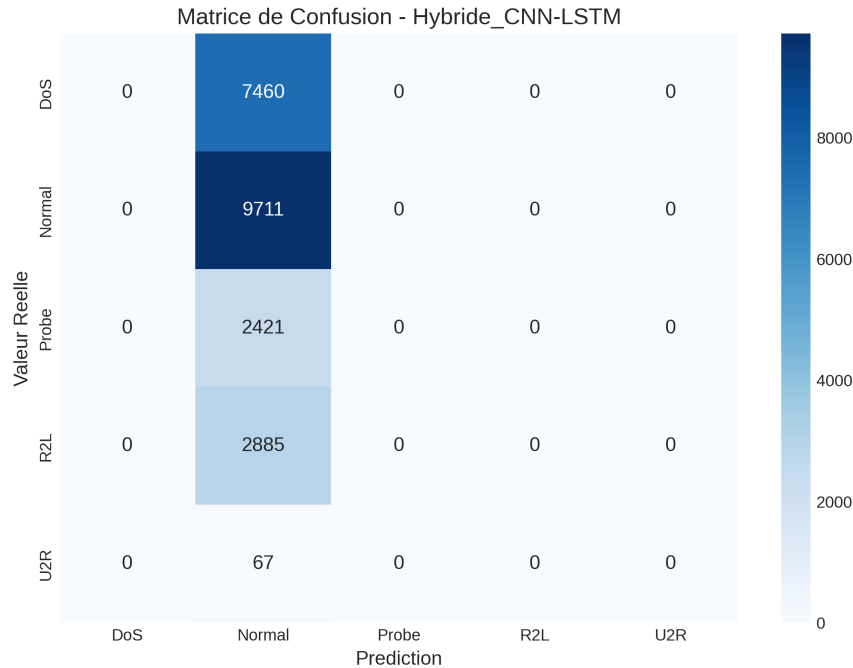


Figure 7.1. Matrice de confusion, Hybride CNN-LSTM (poids non entraînés). Le modèle prédit systématiquement la classe Normal pour toutes les connexions. Ce comportement est caractéristique d'un réseau à poids aléatoires non initialisés : le biais de la couche de sortie pousse vers la classe dominante (Normal représente 43 % du jeu de test). Ces résultats ne reflètent pas les capacités architecturales du modèle.

Le modèle a été évalué avec le flag `-skip-hybrid`, qui charge l'architecture sans procéder à l'entraînement. Les métriques obtenues (Accuracy = 0,431, F1 = 0,259, AUC = 0,564) correspondent à une prédiction quasiment aléatoire et n'ont pas de valeur comparative.

Note, Contrainte matérielle

PyTorch détecte automatiquement CUDA. Sur CPU (AMD Ryzen 9 7900, 12 cœurs), un entraînement complet du CNN-LSTM prendrait plusieurs heures. L'entraînement a été omis dans ce run pour des raisons de temps. Un GPU NVIDIA permettrait de réduire ce temps d'un facteur 10 à 50.

8. Limites et Perspectives

Rédacteur : Thomas David

8.1 Limites

8.1.1 Dataset obsolète

NSL-KDD a été construit en 1999 à partir d'un environnement réseau simulé. Le trafic réseau actuel est fondamentalement différent : généralisation de HTTPS (chiffrement end-to-end), virtualisation, conteneurisation, trafic IoT, services cloud. Les 41 features pré-calculées du dataset ne correspondent pas aux features extractibles à partir de paquets HTTPS. La transposabilité directe de nos modèles vers un environnement de production est donc limitée.

8.1.2 Biais de SMOTE

L'écart entre les scores de validation ($F1 \approx 0,999$) et les scores de test ($F1 \approx 0,72$) est principalement dû à SMOTE. Les exemples synthétiques améliorent la performance sur des données similaires à celles d'entraînement, mais ne reflètent pas la vraie variabilité des attaques rares. Ce biais de validation est documenté dans la littérature et constitue un risque sérieux si l'on ne dispose que d'une métrique de validation pour évaluer le modèle.

8.1.3 Classes rares insuffisamment apprises

Avec 52 exemples U2R d'origine, SMOTE ne peut créer de la variété que là où elle existe. La classe R2L, dont la distribution dans le test est 3 fois plus élevée qu'en entraînement, est mal apprise pour la même raison. Ces deux classes sont précisément celles correspondant aux attaques les plus critiques en termes d'impact.

8.2 Perspectives

8.2.1 Données

Les datasets CICIDS2017 et CSE-CIC-IDS2018 capturent du trafic HTTPS moderne avec extraction de features sur paquets bruts (via CICFlowMeter). Ils offrent une représentation plus fidèle du trafic contemporain et des attaques actuelles (DDoS amplification, cryptomining).

8.2.2 Modèles

Les architectures Transformer (mécanisme d'attention sans récurrence) ont démontré leur supériorité sur les LSTM pour les séquences longues dans de nombreux domaines. Appliqués à des séquences de connexions réseau, ils pourraient mieux capturer les patterns d'attaques multi-étapes.

8.2.3 Déploiement

Un pipeline temps réel avec Zeek ou Scapy pour l'extraction de features, suivi d'une inférence avec les modèles sauvegardés, permettrait une détection avec une latence de l'ordre de la milliseconde. L'intégration avec un SIEM (Splunk, Elastic Security) permettrait d'automatiser la réponse aux incidents.

8.2.4 Contre le déséquilibre

Les GAN (Generative Adversarial Networks) peuvent générer des exemples synthétiques plus réalistes que SMOTE en apprenant la distribution réelle des données minoritaires. Alternativement, le *cost-sensitive learning* pénalise plus fortement les erreurs sur U2R et R2L sans modifier les données.

9. Conclusion

Rédacteur : Thomas David

Ce projet a conduit à l'implémentation d'un pipeline complet de détection d'intrusions réseau basé sur le Machine Learning, couvrant l'acquisition des données, le preprocessing, l'entraînement de cinq modèles classiques, l'architecture d'un modèle Deep Learning hybride, et l'analyse d'interprétabilité.

Les résultats obtenus sur le jeu de test NSL-KDD ($F1 \approx 0,72-0,73$, $AUC-ROC \approx 0,92-0,96$) sont cohérents avec les benchmarks honnêtes de la littérature sur ce problème de classification à 5 classes. L'écart avec les scores de validation SMOTE ($F1 \approx 0,999$) illustre un piège classique de l'évaluation en ML : une performance élevée en validation ne garantit pas une performance équivalente en conditions réelles.

Les enseignements principaux sont les suivants :

1. **XGBoost et LightGBM offrent le meilleur compromis performance/vitesse** sur des données tabulaires structurées. En quelques secondes d'entraînement, ils atteignent les mêmes scores que le MLP (184 s) et le SVM (1 343 s).
2. **Le SVM est à privilégier pour les classes rares.** Son recall U2R de 0,49 est le double de ses concurrents. Ce comportement est structurel : la maximisation de la marge SVM est plus conservatrice que le vote d'ensemble.
3. **SMOTE seul ne résout pas le problème des classes rares.** Le recall R2L reste inférieur à 0,25 pour tous les modèles, malgré l'oversampling.
4. **L'interprétabilité SHAP est indispensable.** Elle confirme que les décisions du modèle reposent sur des features ayant une sémantique claire (`src_bytes`, `dst_host_srv_count`), ce qui valide la pertinence de l'approche et permet à un analyste de sécurité de comprendre et de contester une prédiction.

A. Manuel de déploiement

A.1 Installation via Docker

L'ensemble de l'environnement d'exécution (dépendances, Scikit-Learn, PyTorch, etc.) est conteneurisé pour faciliter le déploiement. Il vous suffit de disposer de Docker installé localement.

```
git clone https://github.com/collapsedbanana/ids-project.git
cd ids_project

# Construction de l'image (1 fois)
docker build -t ids_project .

# Execution du conteneur en mode interactif
docker run -it -v $(pwd)/data:/app/data ids_project
```

A.2 Structure du projet

```
ids_project/
|-- main.py                # Orchestrateur du pipeline
|-- src/
|   |-- download_data.py   # Telechargement NSL-KDD
|   |-- preprocessing.py   # Encodage, normalisation, SMOTE
|   |-- models.py          # RF, SVM, MLP, XGBoost, LightGBM
|   |-- hybrid_model.py    # CNN-LSTM + Attention (PyTorch)
|   |-- evaluation.py      # Metriques, matrices, SHAP
|-- data/raw/              # Dataset NSL-KDD
|-- models/                # Modeles sauvegardes (.joblib)
'-- reports/figures/       # Graphiques generes
```

A.3 Exécution

```
# Sans optimisation, sans CNN-LSTM (~26 min, SVM seul = 22 min)
python main.py --no-optimize --skip-hybrid

# Sans optimisation, avec CNN-LSTM (plusieurs heures sur CPU)
python main.py --no-optimize

# Pipeline complet avec Optuna (~2 h minimum)
python main.py
```

```
# Suivi en temps reel
python main.py --no-optimize --skip-hybrid 2>&1 | tee logs/run.
log
tail -f logs/run.log
```

A.4 Fichiers générés

Chemin	Contenu
data/raw/	Dataset NSL-KDD (téléchargé automatiquement)
models/*.joblib	Modèles entraînés (chargement instantané)
reports/figures/	PNG : matrices de confusion, ROC, SHAP, PCA
reports/figures/comparison_table.csv	Tableau complet des métriques
logs/pipeline.log	Journal d'exécution horodaté